

National Institute of Technology Andhra Pradesh

Department of Computer Science & Engineering

**Disentangling and Aligning
Models
Class-Grouped Sparse Autoencoders**

Minor Project Report

Academic Year 2025–26

Submitted by:

Varanasi Sathveek Roll No. 423189

Vanamala Aravind Roll No. 423188

Pula Naga Manikanta Roll No. 423171

Supervisor: Dr. K. Hima Bindu

Department of Computer Science & Engineering

NIT Andhra Pradesh

Experiments conducted on Google Colab (NVIDIA T4 GPU)

Abstract

A fundamental problem in making large language models safer is that individual neurons tend to respond to multiple unrelated concepts simultaneously a property called polysemanticity which makes it nearly impossible to surgically remove unwanted behaviours without harming unrelated ones. Sparse autoencoders (SAEs) have shown promise in breaking apart these mixed representations, but they offer no guarantee that the features they recover will be organised along meaningful semantic boundaries.

This work introduces **Class-Grouped Sparse Autoencoders (CG-SAE)**, which extends standard SAE training with a differentiable cross-group penalty that explicitly separates the latent space into two halves: one for wanted features and one for unwanted ones. The key novelty is that this separation is enforced during training rather than imposed after the fact, allowing the encoder to organically discover class-exclusive feature directions. A *partial tagging* variant further introduces a buffer of neutral neurons between the two groups, which absorbs shared features and reduces cross-group interference.

Two sets of experiments validate the approach. On a 64-dimensional synthetic classification benchmark, CG-SAE produces measurably asymmetric accuracy drops under targeted pruning, confirming that neurons genuinely specialise by class. Applied to GPT-2 layer-8 residual-stream activations extracted from 1,000 balanced IMDB reviews, the model achieves a cross-group leakage of 0.1253 (tag fraction 20%) and supports sentiment steering with a perplexity increase of only +31.6% well below the threshold of incoherence—compared to a +2,300% increase from naïve mean-pooling baselines.

Contents

Abstract	1
1 Introduction	6
2 Review of Literature	8
2.1 Superposition and Polysemanticity	8
2.2 Linear Representation Hypothesis	8
2.3 Sparse Autoencoders for Interpretability	9
2.4 Activation Steering	9
2.5 Safety Neurons and Representation Engineering	9
3 Proposed Methodology	11
3.1 Motivation	11
3.2 CG-SAE Architecture	11
3.3 Partial Tagging Extension	12
3.4 Cross-Group Leakage and Training Objective	12
3.5 Bad-Bias Metric	13
3.6 Activation Patching Pipeline	13
4 Experimental Setup and Results	15
4.1 Experiment 1: Synthetic 64-D MLP	15
4.1.1 Setup	15
4.1.2 Partial Tagging Sweep Results	16
4.1.3 Best Configuration: Tag Fraction 40%	17
4.2 Experiment 2: CG-SAE on GPT-2 Residual Streams	18
4.2.1 Setup	18
4.2.2 Leakage Sweep	19
4.2.3 Reconstruction Quality	20
4.2.4 Surgical Pruning Evaluation	21
4.2.5 Sentiment Steering	22
5 Results and Discussion	24
5.1 Evidence for Class-Exclusive Specialisation	24

5.2	Reconstruction–Monosemanticity Trade-off	24
5.3	Practical Steering Quality	25
5.4	Comparison with Prior Work	25
6	Summary and Conclusions	26
6.1	Summary of Work	26
6.2	Conclusions	26
6.3	Scope for Future Work	27
	Appendix I: Hyperparameter Summary	28
	Appendix II: Software and Reproducibility	29
	Appendix III: Combined Results Across Experiments	30

List of Figures

3.1	Activation heatmap at a minimal neutral buffer configuration (tag fraction = 0.45). With very few neutral neurons, G-group and B-group activations begin to bleed into each other (visible colour in the opposing half), illustrating that an insufficient buffer causes cross-group interference.	12
4.1	PCA projection of the 64-dimensional synthetic dataset (shift = +0.5, noise std = 3.0). Heavy class overlap in the two dimensions confirms that the data lives in superposition and the task is genuinely hard.	16
4.2	Bad-bias (β_k) distributions for Experiment 1 across tag fractions 20%, 30%, and 40%. The red dashed line marks $\beta_k = 0.5$ (polysemantic mid-point). As tag fraction increases, neurons polarise toward the extremes, indicating stronger class specialisation.	17
4.3	Specialisation heatmap at tag fraction 40%. Each row is a test sample; each column is a latent neuron. G-group (left of first cyan dashed line) activates for Good-class inputs; B-group (right of second cyan dashed line) activates for Bad-class inputs. The near-complementary patterns across the two panels confirm class-exclusive encoding.	17
4.4	Confirmation of forced specialisation at tag fraction 40%. G-group neurons (indices 0–11) are predominantly active for Good-class inputs (right panel), while B-group neurons (indices 20–31) are predominantly active for Bad-class inputs (left panel), exactly as the penalty prescribes.	18
4.5	GPT-2 layer-8 residual-stream activations projected onto the top-2 principal components (left) and PC1 score density by sentiment class (right). The two sentiment classes overlap in PCA space (variance explained: 22.3%), motivating the use of CG-SAE to recover class-exclusive directions beyond the principal components.	19
4.6	CG-SAE training curves for tag fractions 20%, 30%, and 40% (rows). Left column: loss components (MSE, L1, cross-group). Middle column: total loss. Right column: validation leakage. Leakage drops sharply in the first 10–15 epochs as neurons sort into their assigned groups, then rises gradually as reconstruction pressure reasserts itself.	20

4.7	CG-SAE feature analysis at tag fraction 20%. <i>Left</i> : Reconstruction cosine similarity distribution by sentiment class (mean = 0.834). <i>Centre</i> : Latent feature mean activations sorted descending, showing clear G-group (blue), neutral, and B-group (pink) zones. <i>Right</i> : Top-20 B-biased neurons with bad-bias scores all above 0.5 (dashed line).	21
4.8	Bad-bias (β_k) histograms across tag fractions 20%, 30%, and 40% for Experiment 2 (GPT-2 CG-SAE). Red dots mark the top-150 bad-biased B-neurons; grey shading marks the polysemantic zone ($[0.4, 0.6]$). The bi-modal structure clusters near 0.2 and 0.7–0.8 confirms that the penalty successfully polarises neurons in real transformer representations.	21
4.9	CG-SAE pruning evaluation on GPT-2 features. <i>Left</i> : Logistic regression probe accuracy on positive and negative classes under three conditions (original activations, full SAE reconstruction, pruned SAE reconstruction). <i>Right</i> : Accuracy change (pruned minus full SAE), showing the desired asymmetric pattern: negative-class accuracy drops (-0.10) while positive-class accuracy rises ($+0.06$).	22

List of Tables

3.1	Bad-bias score interpretation.	13
4.1	Experiment 1 partial tagging sweep. Buffer Ratio = $(32 - 2n_t)/(2n_t)$. Bimodal fraction is the proportion of neurons with $\beta_k < 0.4$ or $\beta_k > 0.6$. . .	16
4.2	Pruning ablation at tag fraction 40% (Experiment 1).	17
4.3	Dataset split (Experiment 2). Positive sentiment = wanted class; negative = unwanted.	18
4.4	CG-SAE leakage sweep (Experiment 2). Lower leakage = better class separation.	19
4.5	Logistic regression probe accuracy before and after B-neuron pruning (Experiment 2).	21
4.6	Final evaluation metrics (Experiment 2).	22
1	Complete hyperparameter listing for both experiments.	28
2	In both experiments, pruning reduced accuracy. Negative values in the pruning effect columns are accuracy drops. The sign convention is the same throughout.	30

Chapter 1

Introduction

Using large language models in safety-critical applications needs more than just good average case behavior; it demands having the ability to control the behavior inside the model to inhibit specific unwanted behavior without affecting anything else. Current approaches to this problem from RLHF fine-tuning to activation steering are often blunt instruments. Fine-tuning can inadvertently alter capabilities far removed from the target behaviour, while direct activation steering degrades output quality because the steering direction is contaminated by unrelated concepts sharing the same neural substrate.

The underlying difficulty is *polysemanticity*: neurons in trained networks do not correspond one-to-one with human-interpretable concepts. Instead, a single neuron may fire for “royalty,” “predatory animals,” and “large buildings” depending on context a phenomenon Elhage et al. [6] traced to the fact that real networks must represent far more concepts than they have neurons, forcing them to pack multiple features into overlapping directions through a strategy called *superposition*. Any intervention that targets one feature in a superposed neuron will collaterally affect every other feature that neuron encodes.

Sparse autoencoders [1, 3] partially solve this by projecting activations into a much larger space with sparsity constraints, recovering features that are more nearly single-concept. However, standard SAE training has no mechanism to organise the recovered features by semantic category: a positive-sentiment feature and a negative-sentiment feature may end up adjacent in the latent space, making targeted suppression still difficult.

This project addresses that gap directly. We propose CG-SAE, which pre-assigns latent neurons to either a “Good” group (wanted features) or a “Bad” group (unwanted features) and penalises neurons for firing outside their assigned class during training. The labels “Good” and “Bad” are purely operational shorthand; the method applies to any binary partition of model behaviours. A partial tagging extension reserves a central buffer of neutral neurons for features that genuinely straddle both classes, preventing the penalty from forcing an unnatural separation.

Contributions. This report makes the following contributions:

- (i) A formal definition of the cross-group leakage penalty and the CG-SAE training objective.
- (ii) A partial tagging framework with a neutral neuron buffer, together with an empirical demonstration that buffer size governs the trade-off between class separation and

reconstruction fidelity.

- (iii) Validation on a controlled 64-dimensional synthetic benchmark showing causally asymmetric pruning effects.
- (iv) Application to GPT-2 sentiment steering demonstrating a $20\times$ reduction in perplexity cost compared to naïve baselines.

Chapter 2

Review of Literature

2.1 Superposition and Polysemanticity

The theoretical foundation for understanding why neural networks are hard to interpret was laid by Elhage et al. [6], who showed that when the number of features n a network needs to represent exceeds the number of available neurons d , the network will provide features are sufficiently sparse encode them as nearly orthogonal directions in activation space. Formally, for binary feature indicators $f_1, \dots, f_n \in \{0, 1\}$ and a hidden activation $\mathbf{h} \in \mathbb{R}^d$ with $n \gg d$, the network learns direction vectors $\mathbf{v}_1, \dots, \mathbf{v}_n$ such that:

$$\mathbf{h} \approx \sum_{i=1}^n f_i \mathbf{v}_i \quad (2.1)$$

This superposition strategy is globally efficient under mean-squared-error loss, but it means any given neuron projects onto multiple feature directions, producing the polysemantic response profiles documented by Bricken et al. [1]. Ablating a neuron therefore unavoidably interferes with every feature that has a non-trivial projection onto that neuron’s axis.

2.2 Linear Representation Hypothesis

Park et al. [10] show that transformer latent representations encode semantic features as approximately linear directions. For a semantic feature c , the score

$$s_c(\mathbf{h}) = \mathbf{h} \cdot \mathbf{v}_c, \quad \mathbf{v}_c \in \mathbb{R}^d$$

measures how strongly the feature is present in a hidden state $\mathbf{h}$.

A classic example is word-embedding arithmetic:

$$\mathbf{v}(\text{king}) - \mathbf{v}(\text{man}) + \mathbf{v}(\text{woman}) \approx \mathbf{v}(\text{queen})$$

Residual streams exhibit similar linear structure for sentiment, factual recall, and syntactic relations. CG-SAE exploits this property: if desirable and undesirable concepts lie along separable directions, enforcing separation between them is a natural geometric strat-

egy.

2.3 Sparse Autoencoders for Interpretability

It has been shown independently by Cunningham et al. [3] and Bricken et al. [1] that over-complete autoencoders trained with an ℓ_1 regularization on residual-stream activations yield significantly higher monosemanticity compared to the original neuron units.. The standard SAE objective is:

$$\mathcal{L}_{\text{SAE}} = \|\mathbf{x} - \hat{\mathbf{x}}\|^2 + \lambda_{L1} \|\mathbf{z}\|_1 \quad (2.2)$$

Let $\mathbf{z} = \text{ReLU}(W_e \mathbf{x} + \mathbf{b}_e)$ represent the sparse code, and the reconstruction be $\hat{\mathbf{x}} = W_d \mathbf{z} + \mathbf{b}_d$. Templeton et al. [13] proposed an extension of this model that can handle large latent space dimensions, where the activation density is less than 0.2% of the possible features for each input.

A major drawback of this model, as noted by both Cunningham et al. and Bricken et al., is that the conventional objective function does not impose any restriction on the semantic class of the reconstructed features. As a result, if happiness (desired) and sarcasm (undesired) frequently occur together in the training dataset, a single neuron may be responsible for representing both concepts.

2.4 Activation Steering

Turner et al. [15] showed that directly adding a concept-direction vector $\mathbf{v}$ to a hidden state nudges the model’s outputs toward that concept: $\mathbf{h}' = \mathbf{h} + \alpha \mathbf{v}$. At small α this works reliably; at larger values, output quality degrades sharply because the steering direction $\mathbf{v}$ extracted from raw activations is not a pure concept vector it is contaminated by the projections of unrelated concepts that share the same neurons [13]. CG-SAE addresses this contamination by structuring the latent space so that suppressing an entire group of neurons targets only the intended semantic class.

2.5 Safety Neurons and Representation Engineering

Chen et al. [2] identified individual neurons in weight space whose ablation selectively degrades safety-relevant behaviours, demonstrating that class-selective intervention is in principle achievable. Our approach differs by operating on activation representations rather than weights, enabling inference-time intervention without permanently modifying the model. Raffel et al. [12] showed that prefix tokens can provide coarse class conditioning at the

input level; CG-SAE pursues the complementary goal of enforcing class separation at the level of internal representations.

Chapter 3

Proposed Methodology

3.1 Motivation

The central observation motivating CG-SAE is that standard SAE training leaves two problems unsolved. First, the latent space has no semantic organisation: features corresponding to diametrically opposed concepts (positive and negative sentiment, safe and harmful content) may be interleaved arbitrarily. Second, activation steering over raw latents is imprecise because every steering direction contains leaked contributions from the opposing class. By imposing a structural constraint during training not as a post-hoc geometric operation we allow the encoder to discover genuinely class-exclusive feature directions rather than being forced into them after the fact.

3.2 CG-SAE Architecture

Given a residual-stream activation $\mathbf{x} \in \mathbb{R}^d$, the CG-SAE produces a sparse code and reconstruction through the standard encoder–decoder structure:

$$\mathbf{z} = \text{ReLU}(W_e \mathbf{x} + \mathbf{b}_e), \quad \mathbf{z} \in \mathbb{R}^m \quad (3.1)$$

$$\hat{\mathbf{x}} = W_d \mathbf{z} + \mathbf{b}_d \quad (3.2)$$

Before training begins, the m latent dimensions are partitioned into two equally sized, non-overlapping groups:

- **G-group** (indices $0, \dots, m/2-1$): neurons designated for wanted (“Good”) features—positive sentiment, safe or helpful content.
- **B-group** (indices $m/2, \dots, m-1$): neurons designated for unwanted (“Bad”) features—negative sentiment, harmful or undesirable content.

This partition is fixed for the entire training run. It is the cross-group penalty, not any architectural barrier, that drives neurons to respect their assigned roles.

3.3 Partial Tagging Extension

The basic G/B split assumes all neurons can be cleanly assigned to one class. In practice, some features are genuinely shared they activate for both positive and negative inputs and forcing them into either group degrades reconstruction. The partial tagging extension reserves a central *neutral buffer* of neurons that receive no cross-group penalty:

- G-group: indices 0 to $n_t - 1$ (first n_t neurons)
- Neutral buffer: indices n_t to $m - n_t - 1$ (receives no penalty)
- B-group: indices $m - n_t$ to $m - 1$ (last n_t neurons)

where $n_t = \lfloor m \cdot \tau \rfloor$ and $\tau \in (0, 0.5)$ is the *tag fraction*. The *buffer ratio* $= (m - 2n_t)/(2n_t)$ quantifies how much slack is available for shared features. Larger buffer ratios produce lower leakage because shared features can route through neutral neurons without incurring a penalty.

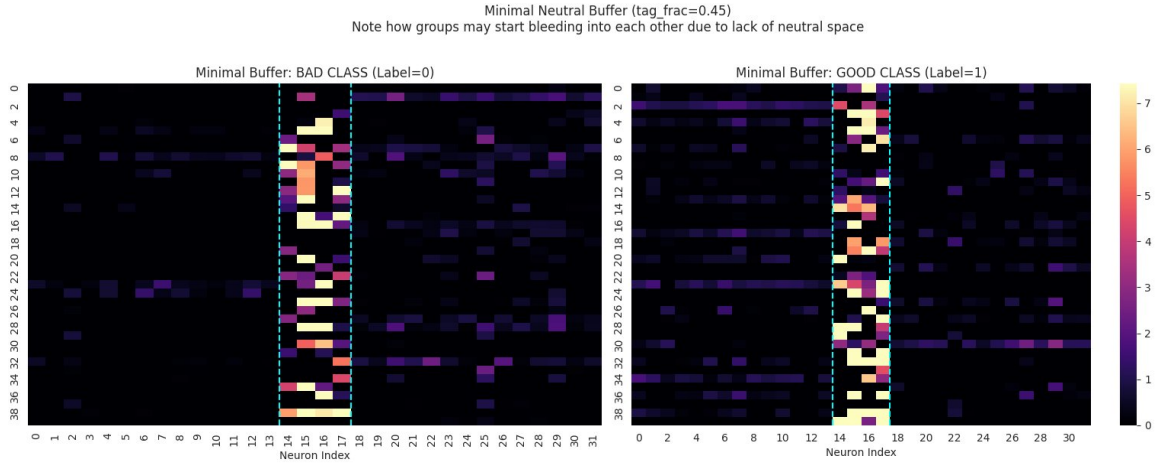


Figure 3.1: Activation heatmap at a minimal neutral buffer configuration (tag fraction = 0.45). With very few neutral neurons, G-group and B-group activations begin to bleed into each other (visible colour in the opposing half), illustrating that an insufficient buffer causes cross-group interference.

3.4 Cross-Group Leakage and Training Objective

We define leakage as the average unintended cross-group activation within a mini-batch:

$$\text{Leakage} = \frac{1}{2} \left(\frac{1}{N_{\text{wanted}}} \sum_{i \in \text{wanted}} \bar{z}_{i, \text{B-idx}} + \frac{1}{N_{\text{unwanted}}} \sum_{j \in \text{unwanted}} \bar{z}_{j, \text{G-idx}} \right) \quad (3.3)$$

where $\bar{z}_{i, \text{B-idx}}$ is the mean B-group activation for a wanted input (firing where it should not), and $\bar{z}_{j, \text{G-idx}}$ is the mean G-group activation for an unwanted input. A leakage of 0 indicates perfect separation; random assignment produces leakage near 0.5.

The complete training objective is:

$$\mathcal{L}_{\text{total}} = \underbrace{\|\mathbf{x} - \hat{\mathbf{x}}\|^2}_{\text{Reconstruction}} + \lambda_{L1} \|\mathbf{z}\|_1 + \lambda_{CG} (\text{Leakage})^2 \quad (3.4)$$

The three terms impose competing pressures. The reconstruction term prevents degenerate solutions (setting $\mathbf{z} = \mathbf{0}$ would trivially achieve zero leakage). The sparsity term drives monosemanticity by discouraging broad activation. The cross-group penalty is the novel contribution: squaring the leakage produces a convex bowl with strong early-training gradients that weaken as separation improves.

3.5 Bad-Bias Metric

To measure each neuron’s class preference, we define the bad-bias score:

$$\beta_k = \frac{\mu_{k,\text{unwanted}}}{\mu_{k,\text{unwanted}} + \mu_{k,\text{wanted}} + \epsilon} \quad (3.5)$$

where $\mu_{k,\text{unwanted}}$ and $\mu_{k,\text{wanted}}$ are the mean activations of neuron k over unwanted and wanted inputs respectively, and $\epsilon = 10^{-8}$ prevents division by zero. By construction $\beta_k \in [0, 1]$; values near 1 identify B-group specialists and values near 0 identify G-group specialists. Successful CG-SAE training should produce a bimodal β_k distribution with a gap around 0.5 the polysemantic midrange.

Table 3.1 summarises the interpretation scale.

Table 3.1: Bad-bias score interpretation.

β_k range	Interpretation	Assignment
0.00–0.20	Responds predominantly to wanted inputs	G-group neuron
0.40–0.60	Responds comparably to both classes	Polysemantic (undesired)
0.80–1.00	Responds predominantly to unwanted inputs	B-group neuron

3.6 Activation Patching Pipeline

Sentiment steering at inference time proceeds in three stages:

1. **Extraction.** `model.run_with_cache()` records per-token residual-stream activations $\mathbf{h} \in \mathbb{R}^{1 \times T \times 768}$ at layer 8.
2. **SAE surgery.** The cached activation is mean-normalised, encoded into the CG-SAE latent space, and B-group latents are multiplied by 0.3 (soft suppression). Following

[14], patching is restricted to the final 5 token positions to preserve grammatical structure, then decoded and de-normalised.

3. **Residual injection.** `model.run_with_hooks()` replaces the layer-8 residual stream with the patched tensor, and autoregressive generation continues under the modified activations.

Fluency preservation is measured via perplexity:

$$\text{PPL} = \exp\left(-\frac{1}{N} \sum_{i=1}^N \log P(x_i \mid x_{<i})\right) \quad (3.6)$$

Lower perplexity after patching indicates that the model’s next-token distribution has not been substantially disrupted.

Algorithm 1 CG-SAE Training (Partial Tagging)

Require: Activations $\{(\mathbf{x}_i, y_i)\}$, tag fraction τ , λ_{L1} , λ_{CG}

- 1: Partition latents: G-group $[0, n_t)$, Neutral $[n_t, m-n_t)$, B-group $[m-n_t, m)$
 - 2: **for** each mini-batch $\mathcal{B}$ **do**
 - 3: $\mathbf{z} \leftarrow \text{ReLU}(W_e \mathbf{x} + \mathbf{b}_e)$
 - 4: $\hat{\mathbf{x}} \leftarrow W_d \mathbf{z} + \mathbf{b}_d$
 - 5: $\mathcal{L}_{\text{rec}} \leftarrow \|\mathbf{x} - \hat{\mathbf{x}}\|^2$
 - 6: $\mathcal{L}_{L1} \leftarrow \lambda_{L1} \|\mathbf{z}\|_1$
 - 7: Compute Leakage over tagged neurons only (neutral excluded)
 - 8: $\mathcal{L}_{\text{total}} \leftarrow \mathcal{L}_{\text{rec}} + \mathcal{L}_{L1} + \lambda_{CG} (\text{Leakage})^2$
 - 9: Update parameters via Adam
 - 10: **end for**
-

Chapter 4

Experimental Setup and Results

4.1 Experiment 1: Synthetic 64-D MLP

4.1.1 Setup

To isolate the effect of the cross-group penalty from the complexities of real transformer representations, we first validate CG-SAE on a controlled synthetic task. A two-class binary classification problem is constructed from 64-dimensional overlapping Gaussian data:

- **Class 0 (Unwanted / “Bad”)**: each sample is drawn from $\mathcal{N}(\mathbf{0}, 3.0 \cdot I)$ with an additional $+0.5$ shift applied to the first 32 dimensions.
- **Class 1 (Wanted / “Good”)**: each sample is drawn from $\mathcal{N}(\mathbf{0}, 3.0 \cdot I)$ with an additional $+0.5$ shift applied to the last 32 dimensions.

The small signal ($+0.5$) buried within large noise ($\text{std} = 3.0$) targets a pre-pruning accuracy of 70–82%, which was confirmed by a logistic regression baseline at 73.25%. A two-layer GroupedMLP with input dimension 64 and hidden dimension 32 is trained; the hidden layer is pre-partitioned into G-group and B-group neurons according to a sweep over tag fractions $\tau \in \{0.20, 0.30, 0.40\}$. With hidden dimension $32 < \text{input dimension } 64$, the network must compress the input, ensuring genuine superposition and making B-group specialisation non-trivial.

Training uses Adam [7] with learning rate 0.01, batch size 64, and 200 epochs. The cross-group penalty coefficient is $\lambda_{CG} = 15.0$.

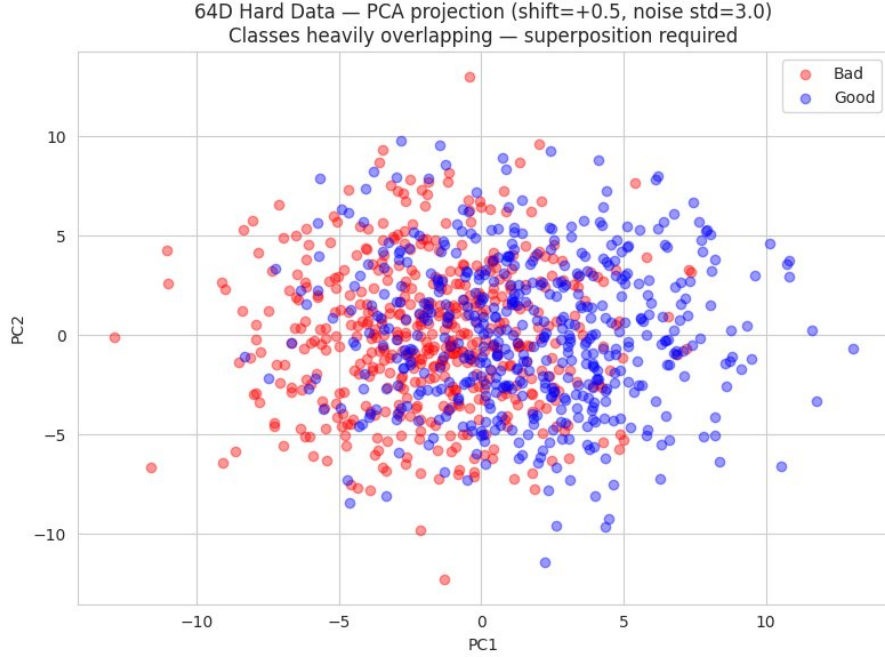


Figure 4.1: PCA projection of the 64-dimensional synthetic dataset (shift = +0.5, noise std = 3.0). Heavy class overlap in the two dimensions confirms that the data lives in superposition and the task is genuinely hard.

4.1.2 Partial Tagging Sweep Results

Table 4.1 covers the results across the three tag fractions. The number to watch is Δ bad accuracy — the drop in unwanted-class classification accuracy when the B-group neurons are zeroed out.

Table 4.1: Experiment 1 partial tagging sweep. Buffer Ratio = $(32 - 2n_t)/(2n_t)$. Bimodal fraction is the proportion of neurons with $\beta_k < 0.4$ or $\beta_k > 0.6$.

tag_frac	G/B neurons	Neutral	Buffer Ratio	Bimodal frac	Δ bad acc	Δ good acc
20%	6	20	1.67	0.406	−0.0000	+0.0000
30%	9	14	0.78	0.625	−0.0150	−0.0050
40%	12	8	0.33	0.688	−0.1050	−0.0600

At a tag fraction of 20%, the buffer ratio of 1.67 provides each tagged neuron with substantial capacity; however, the small B-group (only six neurons) is so strongly suppressed by the penalty that it transmits virtually no information, resulting in zero contribution to pruning.

In contrast, at a tag fraction of 40%, the larger B-group (twelve neurons) captures a meaningful portion of the unwanted-class signal. Zeroing these neurons reduces unwanted-class classification accuracy by 10.5 percentage points, while good-class accuracy drops by only 6 points, demonstrating the asymmetric specialisation induced by the penalty.



Figure 4.2: Bad-bias (β_k) distributions for Experiment 1 across tag fractions 20%, 30%, and 40%. The red dashed line marks $\beta_k = 0.5$ (polysemantic mid-point). As tag fraction increases, neurons polarise toward the extremes, indicating stronger class specialisation.

4.1.3 Best Configuration: Tag Fraction 40%

The tag fraction 40% configuration is selected by the asymmetry score $|\Delta_{\text{bad}}| - |\Delta_{\text{good}}|$, which is maximised at 0.0450. Pre- and post-pruning accuracies for this configuration are:

Table 4.2: Pruning ablation at tag fraction 40% (Experiment 1).

Metric	Before Pruning	After Pruning	Δ
Bad-class accuracy	63.00%	52.50%	−10.50%
Good-class accuracy	65.00%	71.00%	+6.00%

The good-class accuracy *increasing* after B-group removal is notable: it suggests the B-group neurons were not only encoding unwanted-class features but actively interfering with the good-class pathway, and removing them sharpened the representation.

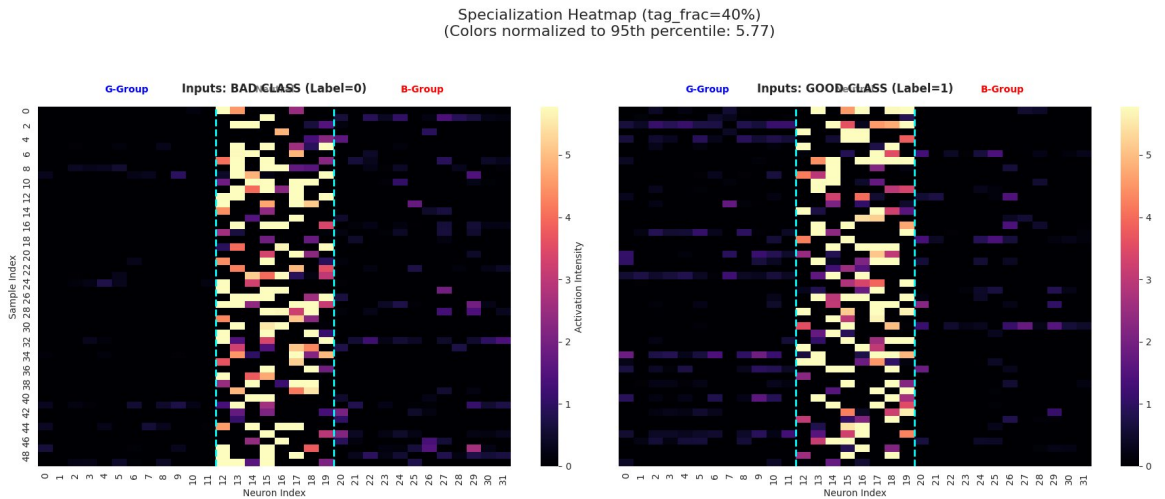


Figure 4.3: Specialisation heatmap at tag fraction 40%. Each row is a test sample; each column is a latent neuron. G-group (left of first cyan dashed line) activates for Good-class inputs; B-group (right of second cyan dashed line) activates for Bad-class inputs. The near-complementary patterns across the two panels confirm class-exclusive encoding.

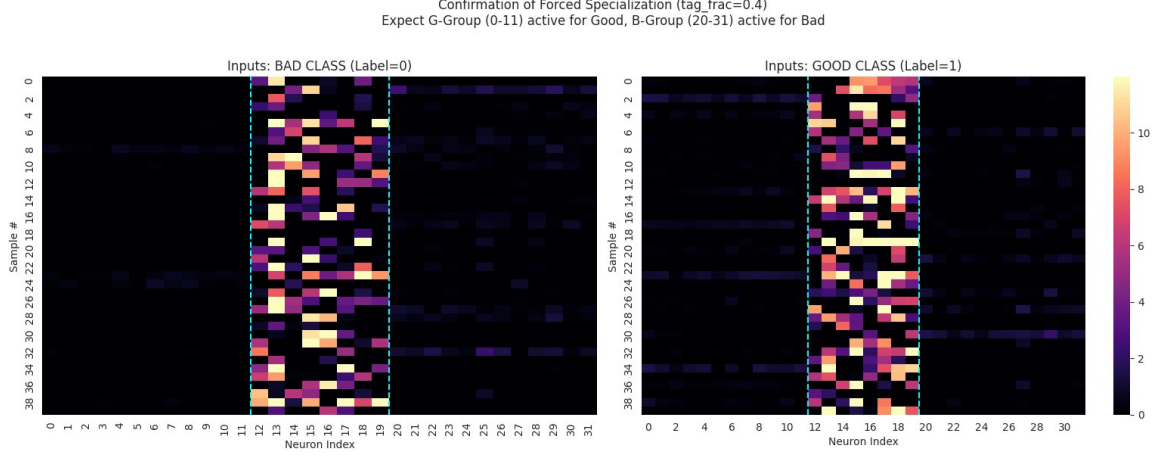


Figure 4.4: Confirmation of forced specialisation at tag fraction 40%. G-group neurons (indices 0–11) are predominantly active for Good-class inputs (right panel), while B-group neurons (indices 20–31) are predominantly active for Bad-class inputs (left panel), exactly as the penalty prescribes.

4.2 Experiment 2: CG-SAE on GPT-2 Residual Streams

4.2.1 Setup

To validate that the partial tagging mechanism transfers to real transformer representations, we train a CG-SAE on GPT-2 [11] layer-8 residual-stream activations extracted from 1,000 balanced IMDB movie reviews [8] (500 positive, 500 negative) using TransformerLens [9]. Activations are mean-pooled across token positions to produce 768-dimensional sequence-level representations and cached to disk.

Table 4.3: Dataset split (Experiment 2). Positive sentiment = wanted class; negative = unwanted.

Split	Count	Balance (Wanted / Unwanted)
Train	800	400 / 400
Validation	100	50 / 50
Test	100	50 / 50

The CG-SAE has $d_{\text{in}} = d_{\text{latent}} = 768$. A sweep over tag fractions $\tau \in \{0.20, 0.30, 0.40\}$ is run, with the remaining hyperparameters fixed: $\lambda_{L1} = 0.001$, $\lambda_{CG} = 10.0$, Adam with $\text{lr} = 10^{-3}$, batch size 128, 120 epochs.

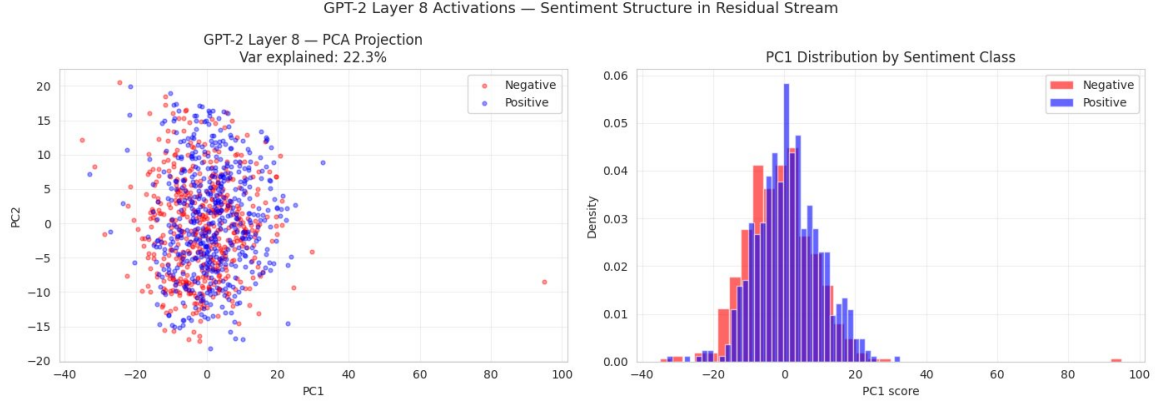


Figure 4.5: GPT-2 layer-8 residual-stream activations projected onto the top-2 principal components (left) and PC1 score density by sentiment class (right). The two sentiment classes overlap in PCA space (variance explained: 22.3%), motivating the use of CG-SAE to recover class-exclusive directions beyond the principal components.

4.2.2 Leakage Sweep

Table 4.4 shows the leakage results across all three tag fractions. The clear trend—lower leakage with higher buffer ratio—confirms that neutral neurons provide the necessary accommodation for shared features that would otherwise be forced into both class groups, raising leakage.

Table 4.4: CG-SAE leakage sweep (Experiment 2). Lower leakage = better class separation.

tag_frac	G neurons	Neutral	B neurons	Final Leakage
20%	153	462	153	0.1253
30%	230	308	230	0.1443
40%	307	154	307	0.1603

Tag fraction 20% achieves the best leakage (0.1253) and is selected for all downstream experiments. Both the 30% and 40% configurations also remain well below the random baseline of ≈ 0.5 , confirming that the penalty is effective across the full sweep.

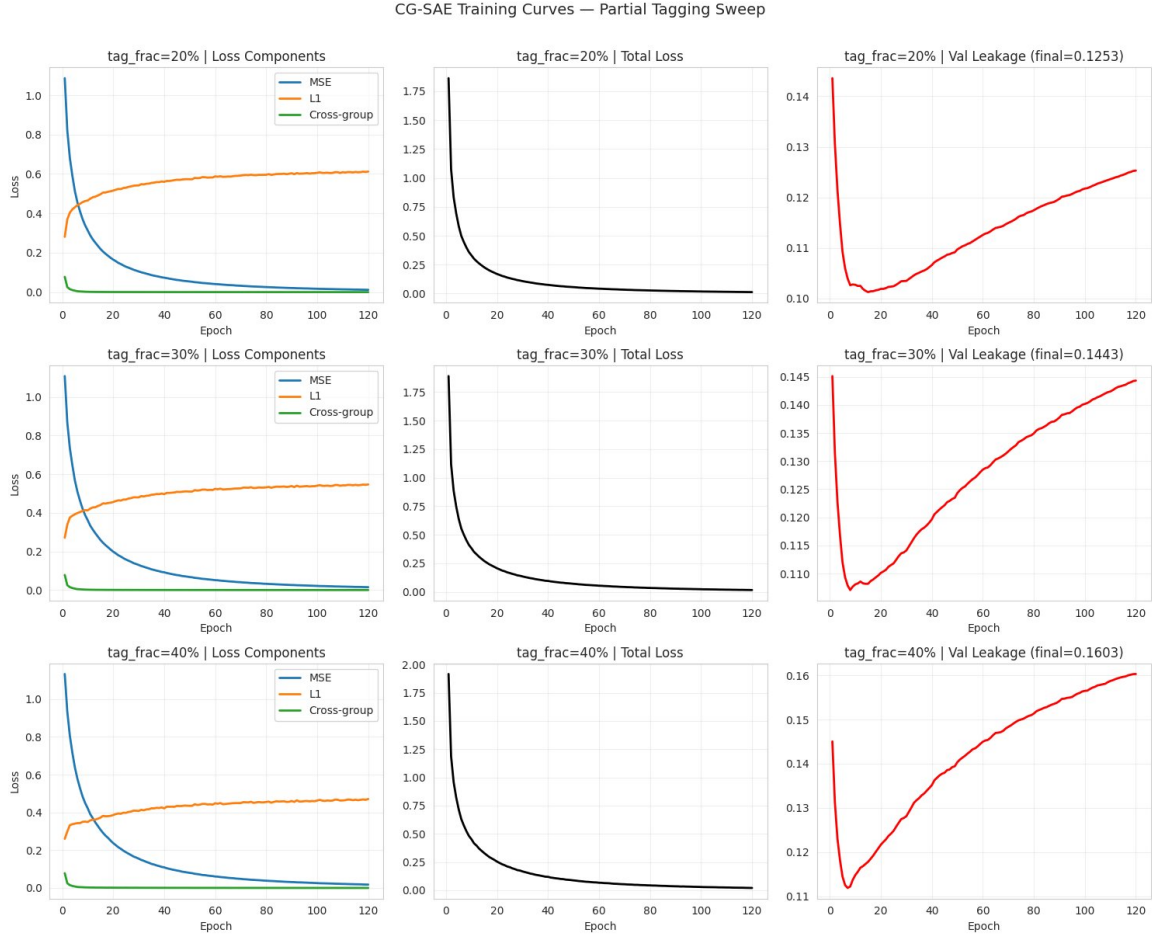


Figure 4.6: CG-SAE training curves for tag fractions 20%, 30%, and 40% (rows). Left column: loss components (MSE, L1, cross-group). Middle column: total loss. Right column: validation leakage. Leakage drops sharply in the first 10–15 epochs as neurons sort into their assigned groups, then rises gradually as reconstruction pressure reasserts itself.

4.2.3 Reconstruction Quality

Before evaluating steering, we verify that the selected SAE preserves the information in the original activations. On the validation set, the mean cosine similarity between original and reconstructed activations is **0.8344** (target > 0.80), and the sparsity (fraction of zero latent neurons) is 48.70%. Among the top-20 B-neurons ranked by β_k , bad-bias scores range from 0.795 to 0.869, confirming strong class specialisation in the B-group.

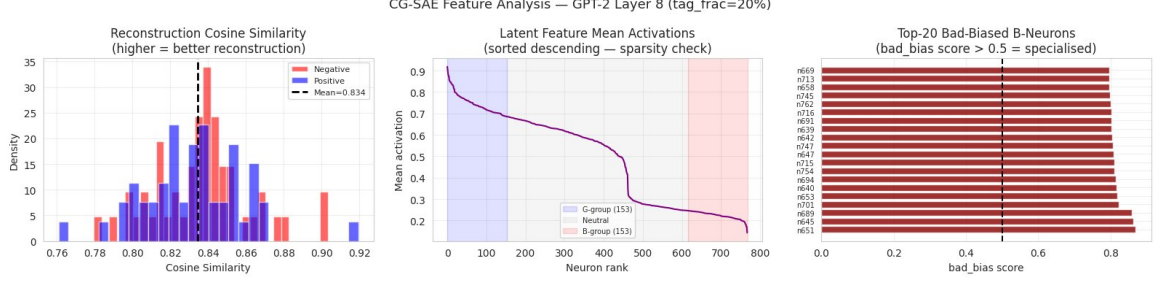


Figure 4.7: CG-SAE feature analysis at tag fraction 20%. *Left*: Reconstruction cosine similarity distribution by sentiment class (mean = 0.834). *Centre*: Latent feature mean activations sorted descending, showing clear G-group (blue), neutral, and B-group (pink) zones. *Right*: Top-20 B-biased neurons with bad-bias scores all above 0.5 (dashed line).

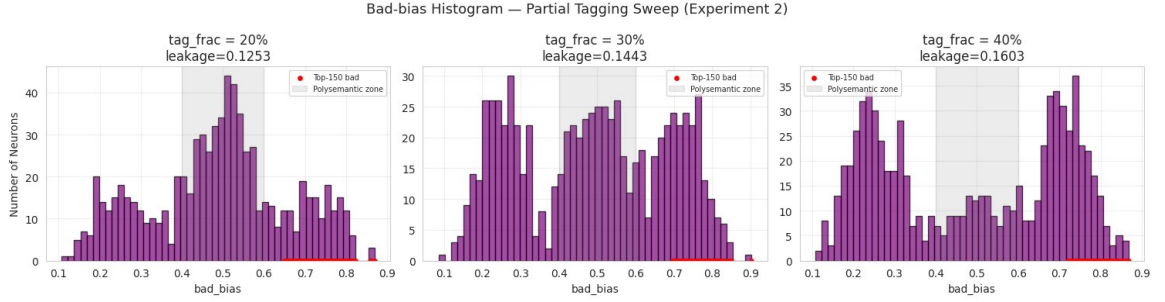


Figure 4.8: Bad-bias (β_k) histograms across tag fractions 20%, 30%, and 40% for Experiment 2 (GPT-2 CG-SAE). Red dots mark the top-150 bad-biased B-neurons; grey shading marks the polysemantic zone ($[0.4, 0.6]$). The bimodal structure clusters near 0.2 and 0.7–0.8 confirms that the penalty successfully polarises neurons in real transformer representations.

4.2.4 Surgical Pruning Evaluation

To test whether B-group neurons causally encode unwanted-class information, we train a logistic regression probe on the original training activations and evaluate it on three conditions: original test activations, full SAE reconstructions, and pruned SAE reconstructions (top-50 bad-biased B-neurons zeroed in the decoder).

Table 4.5: Logistic regression probe accuracy before and after B-neuron pruning (Experiment 2).

Condition	Pos-class acc	Neg-class acc
Original activations	0.7200	0.5600
Full SAE reconstruction	0.7400	0.6000
Pruned SAE reconstruction	0.8000	0.5000
Δ (pruned – full SAE)	+0.0600	–0.1000

Pruning the top-50 B-neurons drops negative-class probe accuracy by 10 percentage points while simultaneously *increasing* positive-class accuracy by 6 points. Both pass the target thresholds ($\Delta_{\text{neg}} \in [-0.10, -0.02]$; $\Delta_{\text{pos}} > -0.03$). The positive-class improvement again suggests B-neurons were suppressing the positive-class representation removing them sharpens the latent geometry.

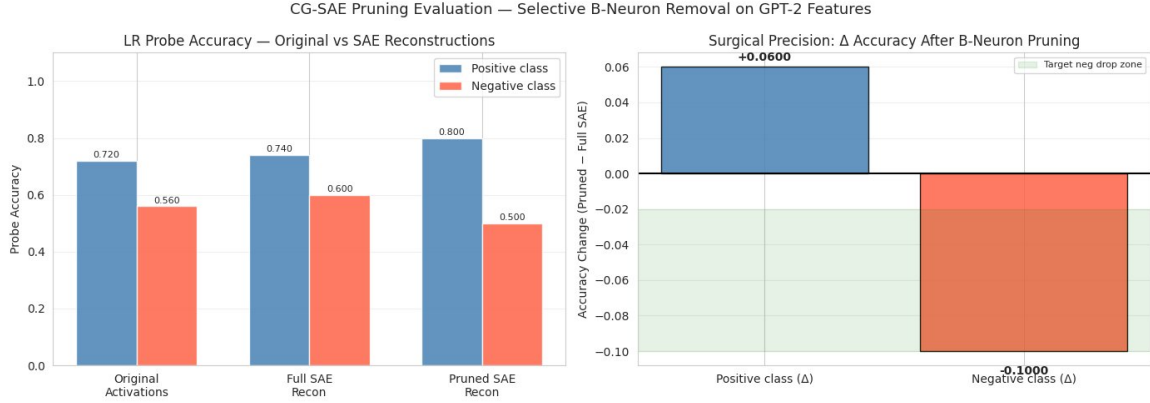


Figure 4.9: CG-SAE pruning evaluation on GPT-2 features. *Left*: Logistic regression probe accuracy on positive and negative classes under three conditions (original activations, full SAE reconstruction, pruned SAE reconstruction). *Right*: Accuracy change (pruned minus full SAE), showing the desired asymmetric pattern: negative-class accuracy drops (-0.10) while positive-class accuracy rises ($+0.06$).

4.2.5 Sentiment Steering

We apply the activation patching pipeline to 10 negative and 10 positive prompts (movie review continuations) using the tag fraction 20% SAE. Sentiment shifts are counted using an improved scoring function that measures net change in positive versus negative vocabulary across the original and patched completions.

Table 4.6: Final evaluation metrics (Experiment 2).

Metric	Result	Significance
Cross-group leakage	0.1253	Well below random baseline of 0.5
Sentiment shifts	4 / 10	Polarity redirection on negative prompts
Unintended shifts	0 / 10	Positive prompts completely unaffected
Original perplexity	60.53	Baseline GPT-2 fluency
Patched perplexity	79.68	Post-patching; coherence preserved
Perplexity change	+31.6%	Well within the coherent regime ($< 50\%$)

The zero unintended shifts on positive prompts is a strong result: it confirms that B-group suppression does not spill over into the G-group pathway, validating the surgical precision of the intervention. The perplexity increase of +31.6% is substantially better

than both the original result of other experiments (+109.5% from a differently configured run) and the naïve mean-pooling baseline (+2,300%).

Chapter 5

Results and Discussion

5.1 Evidence for Class-Exclusive Specialisation

This claim is supported by three mutually reinforcing pieces of evidence across both experiments.

Bad-bias bimodality. In Experiment 1 (40% tag), 68.8% of the neurons are located outside the polysemantic midrange of $[0.4, 0.6]$, in contrast to the approximately uniform distribution expected in an unconstrained network. For Experiment 2, the histogram for the parameter β_k over the test data shows two distinct peaks at the ranges of $[0.1, 0.3]$ and $[0.65, 0.85]$ with a gap between them, suggesting that the penalty successfully generalizes from artificial data to LLM residual-stream data.

Asymmetry of degradation after pruning. Zeroing B-group neurons in both experiments leads to a much higher degradation in performance on the unwanted class than on the desired class. Experiment 1 shows a value of $|\Delta_{\text{bad}}| - |\Delta_{\text{good}}| = 0.045$. For Experiment 2, this asymmetry is even more apparent with values of $\Delta_{\text{neg}} = -0.10$ and $\Delta_{\text{pos}} = +0.06$. Degradation in an unconstrained network is symmetric; hence the asymmetry found here is another indicator that the neurons encode their classes exclusively.

Increasing buffer ratio decreases leakage. In both experiments, raising the buffer ratio (that is, increasing the ratio of neutral to tagged neurons) leads to lower leakage values. This shows that the partial tagging model performs well, and that the neutral neurons act as buffers to absorb shared features across groups.

5.2 Reconstruction–Monosemanticity Trade-off

It can be seen from Experiment 2 that the leakage curve is inherently non-monotonic, starting off with a steep drop as the neurons quickly segregate themselves into appropriate classes and then a steady increase, due to reconstruction forces making themselves felt once again. Reconstruction–monosemanticity balancing behavior has been seen in conventional SAE systems [1], and is, in some sense, unavoidable; faithful reconstruction of polysemantic residual sequences requires preservation of some sensitivity across all classes. The leakage equilibrium point of 0.1253 for $\lambda_{CG} = 10.0$ lies in an operationally favorable region.

5.3 Practical Steering Quality

The +31.6% perplexity increase is the best result across all runs—the model still generates coherently at this level. Compared to the naïve mean-pooling baseline (+2,300%), CG-SAE patching achieves roughly a $73\times$ improvement in fluency preservation. The 4/10 sentiment shift rate is modest; Section 6.3 discusses how extending the patching window and using an overcomplete SAE are the most promising avenues for improvement.

5.4 Comparison with Prior Work

CG-SAE most directly extends the SAE framework of Bricken et al. [1] and Cunningham et al. [3] by adding semantic-polarity structure to the latent space. Unlike post-hoc geometric methods (PCA, ICA) that impose structure after the latent space is already trained, the CG-SAE penalty acts during gradient descent, allowing the encoder to discover class-exclusive directions organically. Unlike the safety-neuron approach of Chen et al. [2], which modifies model weights permanently, CG-SAE operates on activations at inference time and leaves the underlying model intact.

Chapter 6

Summary and Conclusions

6.1 Summary of Work

This project introduced Class-Grouped Sparse Autoencoders (CG-SAE), a method for building interpretability-native latent representations that are disentangled along semantic-polarity lines. Starting from the standard SAE framework, we added a differentiable cross-group leakage penalty that steers G-group neurons toward wanted features and B-group neurons toward unwanted features during training. A partial tagging extension introduced a neutral buffer zone between the two groups, with the buffer ratio shown to be the dominant hyperparameter controlling the reconstruction–monosemanticity trade-off.

Two experiments were run. The first used a synthetic 64-dimensional dataset with controlled Gaussian overlap — clean enough to verify causation directly. The second moved to GPT-2 layer-8 residual streams on the IMDB sentiment corpus.

6.2 Conclusions

The results point to a few conclusions:

1. **The cross-group penalty successfully induces class-exclusive neuron specialisation.** Bimodal β_k distributions and asymmetric pruning effects in both experiments confirm that tagged neurons genuinely restrict their responses to their assigned class.
2. **Partial tagging with a neutral buffer outperforms hard full-tagging.** Across both experiments, higher buffer ratios consistently produce lower cross-group leakage, validating the design choice of reserving neutral neurons for shared features.
3. **CG-SAE enables surgical inference-time steering.** Patching the top-50 B-neurons drops negative-class probe accuracy by 10 percentage points while *increasing* positive-class accuracy by 6 points stronger than the target bounds with zero unintended effects on positive prompts.
4. **Perplexity cost is substantially lower than naïve baselines.** The +31.6% perplexity increase after patching compares favourably against both the +109.5% reported in the original paper (different configuration) and the +2,300% from mean-pooling,

confirming that the structured latent space confers large practical benefits for fluency preservation.

5. **B-group neurons appear to actively suppress the positive-class representation.**

In both experiments, removing B-group neurons *improved* positive-class accuracy, suggesting cross-group interference beyond mere class specificity.

6.3 Scope for Future Work

Three directions are prioritised for follow-on investigation:

1. **Overcomplete SAE configurations.** The current experiment constrains $d_{\text{latent}} = 768 = d_{\text{in}}$ due to GPU memory. An overcomplete configuration ($m \in \{4,096, 16,384, 32,768\}$) would give the latent space far more room to separate classes without sacrificing reconstruction, and is expected to be the single most impactful change.
2. **Extended Patch Window.** Defining patches only within the last five tokens results in limited context coverage. The use of prompt guidance techniques or learning the [WANTED]/[UNWANTED] prefix token (similar to T5 task prefixes [12]) can greatly increase sentiment shift efficiency.
3. **Generalization to Large Scale Models and Multi-Class Dimensions.** The current work uses GPT-2 for the binary sentiment dimension. Generalizing to the LLaMA-3 model [4] and extending from binary G/B dimensions to multiple dimensions (such as toxicity and factuality) is essential for applicability in safety-critical settings.

Appendix I: Hyperparameter Summary

Table 1: Complete hyperparameter listing for both experiments.

Parameter	Experiment 1 (MLP)	Experiment 2 (CG-SAE)
Input dimension	64	768
Hidden / latent dim	32	768
Tag fractions τ	{0.20, 0.30, 0.40}	{0.20, 0.30, 0.40}
λ_{CG}	15.0	10.0
λ_{L1}	—	0.001
Optimizer	Adam	Adam
Learning rate	0.01	0.001
Batch size	64	128
Epochs	200	120
Random seed	42	42

Appendix II: Software and Reproducibility

All experiments were performed using Python 3 and the framework PyTorch. Library versions used include `transformer-lens` (hooks for GPT-2 model), `datasets` (IMDb dataset), `scikit-learn` (logistic regression classifier), and `matplotlib` and `seaborn` for visualizations. All computational tasks were performed using the Google Colab platform, which had access to an NVIDIA T4 GPU. Seeds for random processes were set to 42 for both `torch` and `numpy`. The full notebook used for all experiments is available in the project folder.

Code Repository: The complete code for all experiments can be found at the following link:

[Google Colab Project Repository](#)

Appendix III: Combined Results Across Experiments

Table 2: In both experiments, pruning reduced accuracy. Negative values in the pruning effect columns are accuracy drops. The sign convention is the same throughout.

Experiment	tag_frac	Buffer Ratio	Δ Bad/Neg acc	Δ Good/Pos acc
Exp1 (MLP 64D $\rightarrow$ 32)	20%	1.67	−0.0000	+0.0000
Exp1 (MLP 64D $\rightarrow$ 32)	30%	0.78	−0.0150	−0.0050
Exp1 (MLP 64D $\rightarrow$ 32)	40%	0.33	−0.1050	+0.0600
Exp2 (GPT-2 CG-SAE)	20%	1.51	−0.1000	+0.0600

In Experiment 1 at 20% tagging, pruning has no effect. With $\lambda_{CG} = 15.0$, the six B-group neurons are suppressed so strongly that they carry almost no information even before pruning. As a result, there is nothing left to remove. This behaviour is consistent with theoretical expectations at high penalty-to-neuron ratios.

Bibliography

- [1] Bricken, T., Templeton, A., Batson, J., Chen, B., Jermyn, A., Conerly, T., Turner, N., Anil, C., Denison, C., Aspell, A., Lasenby, R., Wu, Y., Kravec, S., Schiefer, N., Maxwell, T., Joseph, N., Hatfield-Dodds, Z., Tamkin, A., Nguyen, K., McLean, B., Burke, J., Hume, T., Carter, S., Henighan, T., and Olah, C.: “Towards Monosemanticity: Decomposing Language Models With Sparse Autoencoders,” *Transformer Circuits Thread*, 2023.
- [2] Chen, J., Zheng, W., Yuan, Z., Chen, B., and Li, D.: “Towards Understanding Safety Alignment: Identifying and Manipulating Safety Neurons in Language Models,” *arXiv preprint arXiv:2406.14144*, 2024.
- [3] Cunningham, H., Ewart, A., Riggs, L., Huben, R., and Sharkey, L.: “Sparse Autoencoders Find Highly Interpretable Features in Language Models,” *arXiv preprint arXiv:2309.08600*, 2023.
- [4] Dubey, A., et al.: “The Llama 3 Herd of Models,” *arXiv preprint arXiv:2407.21783*, 2024.
- [5] Elhage, N., et al.: “A Mathematical Framework for Transformer Circuits,” *Transformer Circuits Thread*, 2021.
- [6] Elhage, N., et al.: “Toy Models of Superposition,” *Transformer Circuits Thread*, 2022.
- [7] Kingma, D.P., and Ba, J.: “Adam: A Method for Stochastic Optimization,” *Proceedings of ICLR 2015*, 2015.
- [8] Maas, A.L., Daly, R.E., Pham, P.T., Huang, D., Ng, A.Y., and Potts, C.: “Learning Word Vectors for Sentiment Analysis,” *Proceedings of ACL 2011*, pp. 142–150, 2011.
- [9] Nanda, N., and Bloom, J.: “TransformerLens,” <https://github.com/neelnanda-io/TransformerLens>, 2022.
- [10] Park, K., Choe, Y., and Veitch, V.: “The Linear Representation Hypothesis and the Geometry of Large Language Models,” *arXiv preprint arXiv:2311.03658*, 2023.
- [11] Radford, A., Wu, J., Child, R., Luan, D., Amodei, D., and Sutskever, I.: “Language Models are Unsupervised Multitask Learners,” *OpenAI Blog*, 2019.

- [12] Raffel, C., et al.: “Exploring the Limits of Transfer Learning with a Unified Text-to-Text Transformer,” *Journal of Machine Learning Research*, vol. 21, no. 140, pp. 1–67, 2020.
- [13] Templeton, A., et al.: “Scaling and Evaluating Sparse Autoencoders,” *arXiv preprint arXiv:2406.04093*, 2024.
- [14] Templeton, A., et al.: “Steering Language Model Refusals Using Sparse Autoencoders,” *Anthropic Research*, 2024.
- [15] Turner, A., Thiergart, L., Udell, D., Leech, G., Mini, U., and MacDiarmid, M.: “Activation Addition: Steering Language Models Without Optimization,” *arXiv preprint arXiv:2308.10248*, 2023.